

Filling the grid

Six years ago while building a goofy side project I faced the following problem: how do I fit 48 squares on a screen so that they're as big as possible? I had just started high school and was programming to procrastinate on my math homework, hence I shamelessly copied a (partly wrong) solution off of StackOverflow and moved on.

There was only code – no correctness proof, no complexity analysis. Having just repeated and passed the first Basisprüfungsblock (finally), I learned with pain that this does not fly at ETH. Using my new hard earned knowledge I set out to make them myself, procrastinating this time on my math homework with... math.

Let's state the problem: find a $b \times a \in \mathbb{N}$ grid (rows x columns) into a $w \times h \in \mathbb{R}^+$ rectangle which fits $n \in \mathbb{N}$ squares such that the side length of the squares $s \in \mathbb{R}^+$ is maximized and the grid does not overflow the rectangle. Since the side length is maximized, the grid will fit in the rectangle's full width or height, or both if a perfect fit is possible. Formally:

$$\begin{aligned}s_w &:= \max \{ s \in \mathbb{R}^+ \mid \exists a, b \in \mathbb{N} . n \leq ab \wedge as = w \wedge bs \leq h \} \\s_h &:= \max \{ s \in \mathbb{R}^+ \mid \exists a, b \in \mathbb{N} . n \leq ab \wedge as \leq h \wedge bs = h \} \\s &:= \max \{ s_w, s_h \} .\end{aligned}$$

s_w is the *fit-width* solution, s_h the *fit-height* solution. The final solution s is the desired maximal side length. Our target is to build an algorithm to find s .

The solution statement as it stands juggles a lot of variables. We can rewrite the grid dimension constraints using only a, b, w, h :

$$\begin{aligned}as_w = w \wedge bs_w \leq h &\iff a \geq \frac{w}{h}b \wedge s_w = \frac{w}{a} \\bs_h = h \wedge as_h \leq w &\iff b \geq \frac{h}{w}a \wedge s_h = \frac{h}{b} .\end{aligned}$$

s is now dependent on the grid and rectangle dimensions. Using $r := \frac{w}{h}$, the rectangle's ratio:

$$\begin{aligned}a_w &:= \min \{ a \in \mathbb{N} \mid \exists b \in \mathbb{N} . n \leq ab \wedge a \geq rb \} \\b_h &:= \min \left\{ b \in \mathbb{N} \mid \exists a \in \mathbb{N} . n \leq ab \wedge b \geq \frac{a}{r} \right\} \\s &= \max \left\{ \frac{w}{a_w}, \frac{h}{b_h} \right\} .\end{aligned}$$

a_w is the column count of the *fit-width* grid, b_h the row count of the *fit-height* grid. They are minimized since the side length is inversely proportional. The side length is a trivial result of the grid dimensions; only the algorithm for finding a_w and b_h remains to be developed.

Let's take a look at a fit-width grid with dimensions $(a_w, b_w) \in \mathbb{N}^2$. The row count b_w can be abstracted away, too:

$$\begin{aligned} n \leq a_w b_w \wedge a_w \geq r b_w &\iff \frac{n}{a_w} \leq b_w \leq \frac{a_w}{r} \\ &\iff \left\lceil \frac{n}{a_w} \right\rceil \leq \frac{a_w}{r} \\ &\iff a_w \geq r \left\lceil \frac{n}{a_w} \right\rceil. \end{aligned}$$

With an analogous derivation for b_h , we get:

$$a_w = \min \left\{ a \in \mathbb{N} \mid a \geq r \left\lceil \frac{n}{a} \right\rceil \right\}, \quad b_h = \min \left\{ b \in \mathbb{N} \mid r b \geq \left\lceil \frac{n}{b} \right\rceil \right\}.$$

Since the second comparison term is non-increasing in a (respectively in b), it follows that every $a > a_w$ (and $b > b_h$) would satisfy the solution condition. The algorithm idea is therefore the following: starting from an initial guess $a_0 \leq a_w$ (and $b_0 \leq b_h$) test every value until the solution condition holds, i.e. loop while:

$$a < r \left\lceil \frac{n}{a} \right\rceil, \quad r b < \left\lceil \frac{n}{b} \right\rceil$$

Only the initial guesses a_0 and b_0 must be established. From the solution condition of a_w :

$$a_w \geq r \left\lceil \frac{n}{a_w} \right\rceil \implies a_w^2 \geq r n \implies a_w \geq \sqrt{r n}$$

Since $a_w \in \mathbb{N}$ it can therefore be no smaller than $a_0 = \lceil \sqrt{r n} \rceil$. Likewise, $b_w \geq b_0 = \lceil \sqrt{\frac{n}{r}} \rceil$. Using the loop condition above, an algorithm can finally be built:

```
function fillGrid(n, w, h) {
  const r = w / h

  let a = Math.ceil(Math.sqrt(r * n))
  while (a < r * Math.ceil(n / a)) {
    a++
  }

  let b = Math.ceil(Math.sqrt(n / r))
  while (r * b < Math.ceil(n / b)) {
    b++
  }

  return Math.max(w / a, h / b)
}
```

Correctness is proven in the construction process above. To analyze the runtime complexity, observe that since $a_0 \leq a < r \left\lceil \frac{n}{a} \right\rceil$, a will be incremented at most $\left\lceil r \left\lceil \frac{n}{a} \right\rceil \right\rceil - a_0$ times. Moreover, since a increases, $\left\lceil \frac{n}{a} \right\rceil \leq \left\lceil \frac{n}{a_0} \right\rceil$. Hence an upper bound on the iteration count is:

$$\begin{aligned}
 \left\lceil r \left\lceil \frac{n}{a} \right\rceil \right\rceil - a_0 &< r \left\lceil \frac{n}{a_0} \right\rceil - a_0 + 1 \\
 &< r \frac{n}{a_0} + r - a_0 + 1 \\
 &= r \frac{n}{\left\lceil \sqrt{rn} \right\rceil} + r - \left\lceil \sqrt{rn} \right\rceil + 1 \\
 &< \frac{rn}{\sqrt{rn}} + r - \sqrt{rn} + 1 \\
 &= r + 1
 \end{aligned}$$

This means at most $\lfloor r \rfloor + 1$ iterations and a runtime of $\mathcal{O}(r)$, dependent only on the aspect ratio of the grid. This bound is tight: for $n = 33, r = \frac{1}{8.3}$ the fit-width loop does $1 = \lfloor r \rfloor + 1$ iteration, reaching the upper bound.

Lastly, *geht es besser?* Yes – the algorithm can be adapted to use binary search, achieving $\mathcal{O}(\log r)$ (how? left to the reader). *Geht es (noch) besser?* Probably not. This problem is a particular case of *integer programming*: find minimal/maximal integer solution to a problem satisfying constraints. It is an NP-hard problem, with state of the art algorithms having complexity $(\log n)^{\mathcal{O}(n)} \cdot (m \cdot \log V)^{\mathcal{O}(1)}$, where:

- n is the number of variables,
- m the number of constraints,
- V the maximum absolute value of the solution.

For our algorithm we'd solve for the fit-width and fit-height solution separately. Handling fit-width: $n = 1$ since we just solve for a_w , $m = 2$ and $V = r$ since by the upper bound above $a_0 \leq a \leq a_0 + r$. Plugging in the formula above, we'd get a runtime of $(2 \log r)^{\mathcal{O}(1)}$, which is not asymptotically better than the binary search solution. The solution developed here is heuristics-based: it does a local search on the solution space, exploiting characteristics unique to this particular problem. The only way to reduce the asymptotic runtime is with a better heuristic.

We have a correct and optimal algorithm, therefore this presentation concludes here.

But there is more to explore:

- can a better heuristic be derived (perhaps even $\mathcal{O}(1)$)?
- can one find geometric intuition behind all this?
- are the grid dimensions unique for the optimal solution? (also an exercise for the reader [WIP: LINK](#))
- as r changes, when are a_w and b_h equal to a_0 and b_0 and when does the algorithm iterate instead?

For the curious I've prepared a deep dive into all of the above in this nice document [WIP: LINK](#).
Alternatively, archive.quateo.com/grid/ hosts all referenced files.

Thank you for reading!